TP3: CNNs para clasificar imágenes satelitales (EuroSAT RGB)

Tecnología Digital VI, UTDT, 2do semestre 2026 · Manuel Ferrario, Iñaki Irarrazaval, Lisandro Russo

En este trabajo clasificamos las 27.000 imágenes de EuroSAT RGB (64 × 64 píxeles, 10 clases de uso del suelo) con dos enfoques: una CNN propia entrenada desde cero y una ResNet18 preentrenada en ImageNet. Todo el código, las salidas y el análisis detallado están en `tp3.ipynb`. Este informe resume qué hicimos, qué decisiones tomamos y qué encontramos.

1. Datos

Usamos la división que trae el dataset: 18.900 imágenes de train (70%), 5.400 de validation (20%) y 2.700 de test (10%). Antes de modelar verificamos que no hubiera archivos ni imágenes con exactamente los mismos píxeles repetidos entre splits, para descartar data leakage por copias exactas. Las clases tienen un desbalance leve (Pasture es la más chica, con 1.400 imágenes en train contra 2.100 de las más grandes) y la división es estratificada, así que usamos accuracy como métrica principal, pero siempre acompañada de precision, recall y F1 por clase.

En el EDA vimos que el color solo no alcanza para separar varias clases: por ejemplo, AnnualCrop y PermanentCrop, o Pasture y River, tienen distribuciones de color muy superpuestas, así que la red tiene que aprender textura y forma. También marcamos de entrada los pares que se parecen a simple vista (entre ellos Highway y River, Industrial y Residential, y Pasture y HerbaceousVegetation), y varios terminaron entre los que más se confunden.

2. Red propia entrenada desde cero

Pipeline. Escalamos los píxeles a [0, 1] y normalizamos cada canal con la media y el desvío calculados solo sobre train. Armamos los `DataLoader` con batches de 64 imágenes. Como data augmentation aplicamos espejos y rotaciones en múltiplos de 90°: en una imagen satelital no hay un "arriba", así que un bosque rotado sigue siendo un bosque. Elegimos solo múltiplos de 90° para no generar bordes negros. El augmentation se aplica solo en train.

Arquitectura. Tres bloques Conv 3×3 + ReLU + MaxPool (32, 64 y 128 filtros), que llevan la imagen de 3 × 64 × 64 a 128 × 8 × 8. Después, una capa lineal de 8.192 a 128 con ReLU, que es el espacio latente que analizamos en la sección 3, y una capa lineal final de 128 a 10. En total tiene 1.143.242 parámetros.

Entrenamiento. Usamos Cross-Entropy y Adam con learning rate 0.001. Primero entrenamos 15 épocas. Como la loss de validation seguía bajando, reentrenamos con 25 épocas y nos quedamos con los pesos de la época con mejor accuracy de validation (early stopping): la época 22, con 0.952 de accuracy y 0.142 de loss. Después de la época 22, validation oscila sin una caída sostenida, por lo que no vemos un overfitting claro, aunque la brecha con train empieza a abrirse.

Gradientes. Durante las 3 primeras épocas guardamos la norma del gradiente de cada capa en cada batch. No hay vanishing: Conv 1, la capa más lejana a la salida, tiene normas promedio de 4.2, 5.6 y 6.0 por época, y solo es baja en los primeros batches (mínimo 0.025). Tampoco hay exploding: la norma máxima es 26.7 y no crece sin control. Es consistente con que la red tiene pocas capas con pesos (5) y usa ReLU.

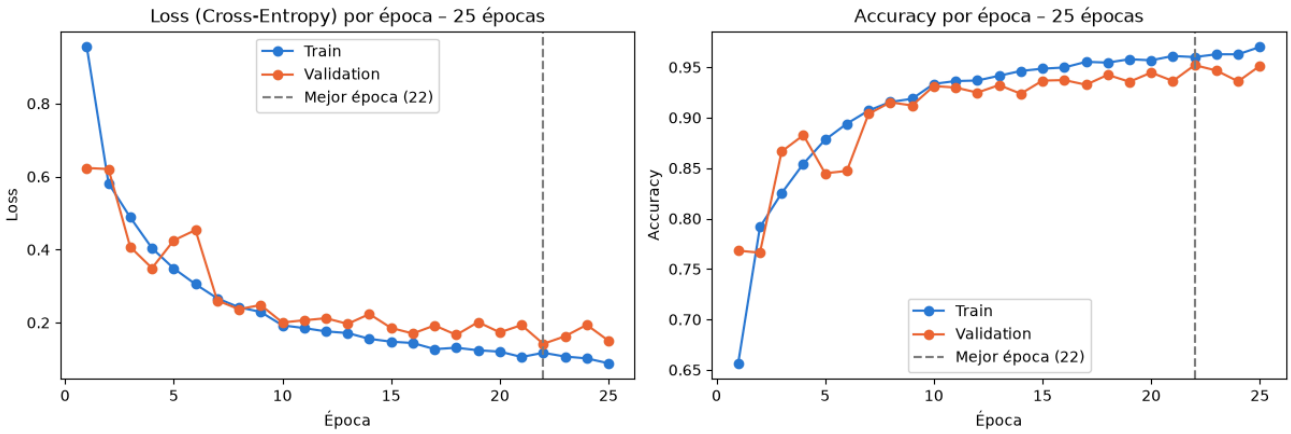


Figura 1. Loss y accuracy de la red propia (25 épocas). La línea marca la mejor época (22).

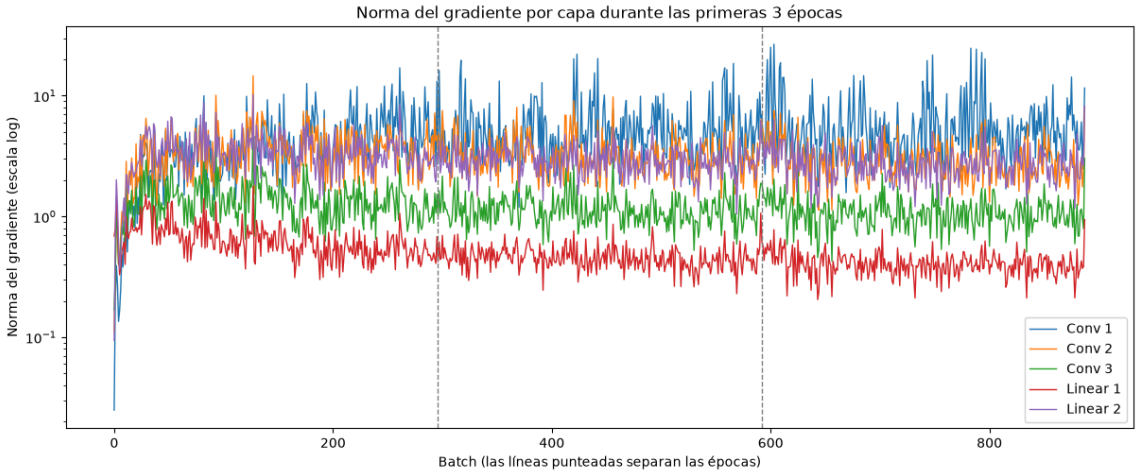


Figura 2. Norma del gradiente por capa en las 3 primeras épocas.

3. Análisis de errores y espacio latente

Test. Evaluamos la red propia en test una sola vez: accuracy 0.955 y F1 macro 0.953, con 122 errores en 2.700 imágenes. Es casi igual al accuracy de validation (0.952), así que elegir la época mirando validation no infló mucho el resultado. Las mejores clases son Forest, Residential y SeaLake (F1 entre 0.985 y 0.987). La peor es Pasture (F1 0.927), que tiene buena precision (0.967) pero el recall más bajo (0.890): 22 de sus 200 imágenes se clasifican como River (6), HerbaceousVegetation (6) o AnnualCrop (5).

Confusiones más frecuentes. Los errores se concentran en pocos pares: los dos primeros ya suman el 30% del total.

- PermanentCrop → HerbaceousVegetation (15 errores, 6% de PermanentCrop). En las imágenes mal clasificadas no se distinguen las hileras de viñedos o frutales. A 10 m por píxel esas hileras pueden quedar por debajo de la resolución, y la parcela se ve como vegetación sin estructura.
- Highway → River (13 errores, 5.2% de Highway). Las dos clases son una línea larga que cruza la imagen, y en varias de las imágenes mal clasificadas el fondo es azulado u oscuro, así que una ruta se parece a un río angosto.
- Industrial → Residential (6 errores). Las Industrial mal clasificadas tienen edificios chicos y amontonados, más parecidos a un barrio que a una zona de galpones.

Espacio latente. Extrajimos el vector de 128 valores que la red calcula antes de la última capa y lo proyectamos a 2D con PCA y con t-SNE (sobre las 50 primeras componentes del PCA). En el PCA las clases están bastante encimadas, porque es una proyección lineal. En el t-SNE, en cambio, la mayoría de las clases forman grupos separados. Los grupos que quedan pegados son justamente los pares que más

se confunden: Highway con River y PermanentCrop con HerbaceousVegetation. Pasture aparece partida en dos, con un grupo chico cerca de River y AnnualCrop. O sea que las capas convolucionales ya ordenan las imágenes por clase, a la capa final le alcanza con trazar fronteras entre esos grupos, y los errores aparecen en esas fronteras.

Como un gráfico de t-SNE puede engañar, lo comprobamos con números. Un kNN (que clasifica cada imagen de test según sus 10 vecinos más cercanos de train) acierta 0.957 usando el espacio latente, casi lo mismo que la red completa, pero solo 0.346 usando los píxeles crudos: en los píxeles, imágenes de la misma clase no están cerca entre sí, y después de pasar por la red sí. La silueta, que mide si cada imagen está más cerca de su clase que de la clase vecina (de -1 a 1), pasa de -0.108 con los píxeles a 0.228 con el espacio latente: hay grupos, aunque no perfectamente separados. Por clase, la silueta es alta en Forest (0.492) y Residential (0.416) y casi 0 en Highway (0.035), HerbaceousVegetation (0.056) y River (0.068), justamente las clases de las dos confusiones principales. Además, repetimos el t-SNE con otra perplexity (50) y los grupos se mantienen.

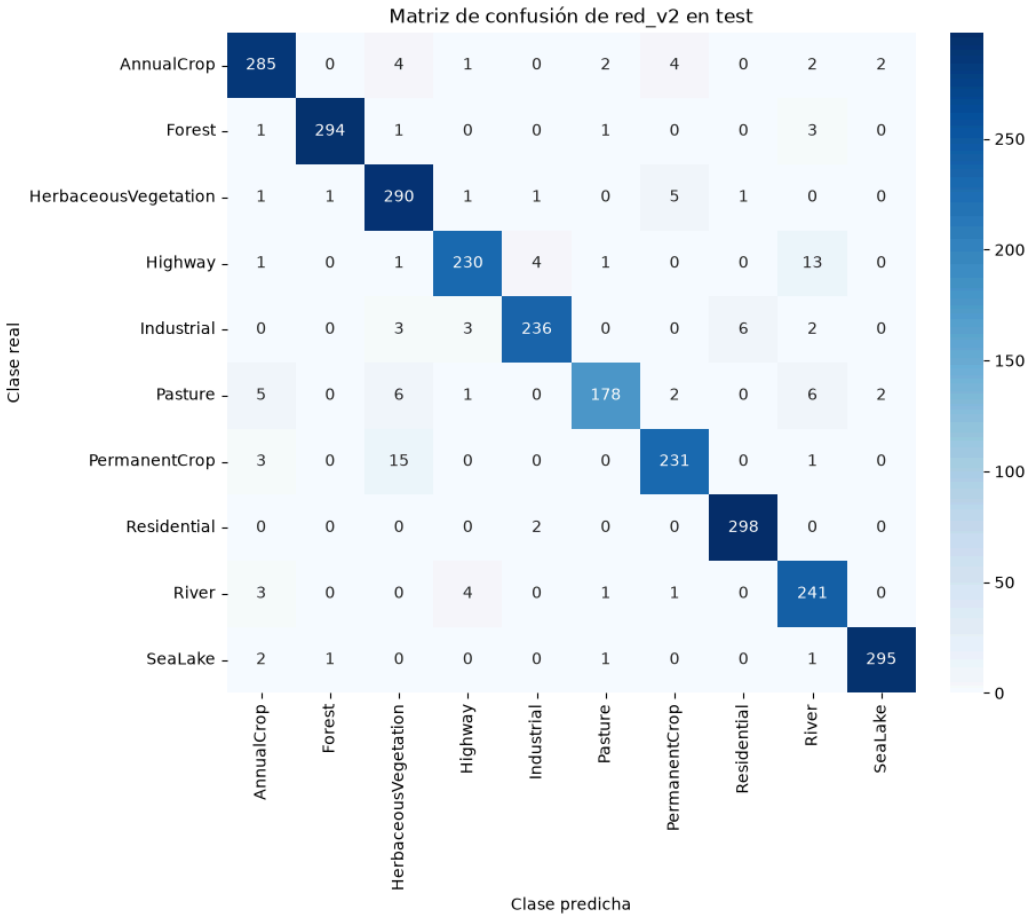


Figura 3. Matriz de confusión de la red propia en test.

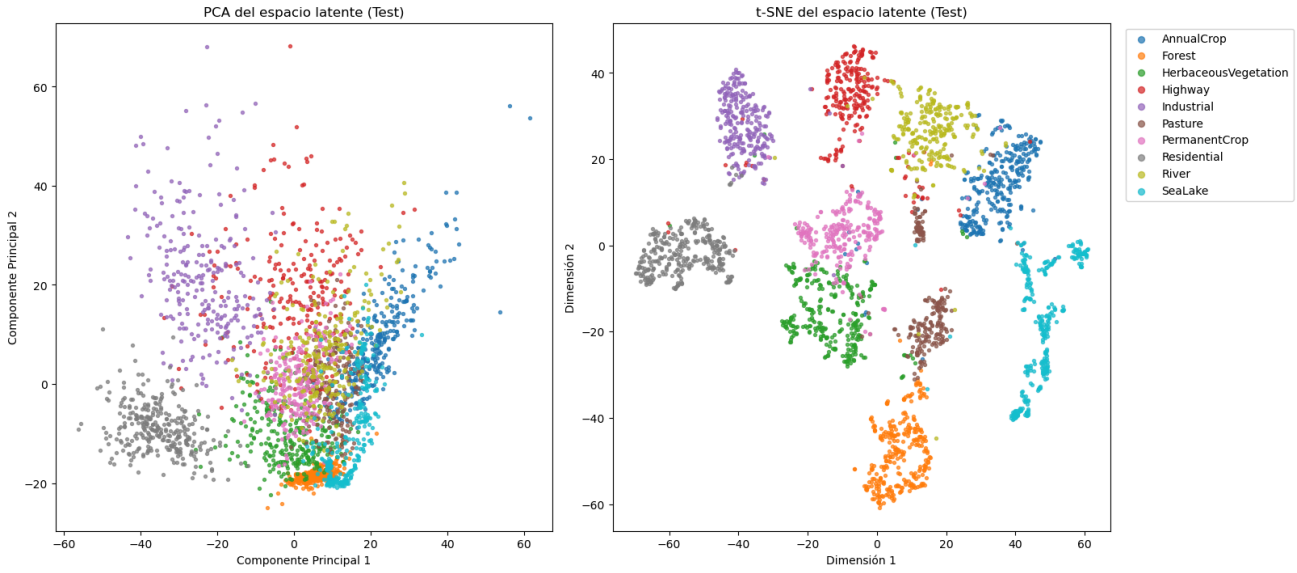


Figura 4. Espacio latente de la red propia en test, proyectado con PCA (izquierda) y t-SNE (derecha).

4. Transfer learning con ResNet18

Elegimos ResNet18 porque es mucho más liviana que VGG16 (11 contra 138 millones de parámetros), lo que importa porque entrenamos en CPU, y porque sus conexiones residuales evitan el vanishing gradient aunque la red sea profunda. Para adaptarla, normalizamos las imágenes con la media y el desvío de ImageNet, que es la escala con la que la red aprendió. También congelamos toda la red, descongelamos el último bloque (`layer4`) y reemplazamos la capa final por una lineal de 512 a 10. Así entrenamos 8.4 de sus 11.2 millones de parámetros, con Adam, un learning rate de 0.0001 (diez veces más chico, para no romper lo que ya aprendió) y el mismo augmentation. Entrenamos 15 épocas y nos quedamos con la mejor según validation (la 14, con 0.963).

	Red propia	**ResNet18**
Parámetros totales / entrenables	1.143.242 / 1.143.242	11.181.642 / 8.398.858
Accuracy de validation después de 1 época	0.768	0.931
Primera época con accuracy de validation ≥ 0.90 / ≥ 0.95	7 / 22	1 / 3
Mejor accuracy de validation (época)	0.952 (22)	0.963 (14)
Accuracy en test / F1 macro en test	0.955 / 0.953	0.965 / 0.963
Errores en test (de 2.700)	122	94
Tiempo por época (misma computadora)	36.0 ± 0.4 s	137.5 ± 6.8 s
Tiempo estimado hasta accuracy de validation ≥ 0.95	13.2 min	6.9 min

Velocidad de convergencia. ResNet18 converge mucho más rápido en épocas: después de una sola época ya tiene 0.931 en validation, y llega a 0.95 en la época 3, contra la 22 de la red propia. Es lo que esperábamos, porque no arranca de cero: ya sabe detectar bordes y texturas, y solo tiene que aprender a usarlos para nuestras clases.

Rendimiento final. ResNet18 gana por un punto en accuracy y en F1 macro de test, con 28 errores menos. Lo que más mejora es Pasture (recall de 0.890 a 0.955), y desaparece la confusión de Industrial con Residential. Pero el par Highway/River sigue siendo el más confundido (19 errores contra 17), solo que ahora en la dirección contraria: ResNet18 confunde más ríos con autopistas.

F1 en test por clase	**Red propia**	**ResNet18**
AnnualCrop	0.948	0.966

Forest	0.987	0.990
HerbaceousVegetation	0.935	0.962
Highway	0.939	0.934
Industrial	0.957	0.972
Pasture	0.927	0.953
PermanentCrop	0.937	0.932
Residential	0.985	0.992
River	0.929	0.941
SeaLake	0.985	0.993

¿Es una diferencia real? Con bootstrap (2.000 sorteos con reposición de las imágenes de test, evaluando las dos redes sobre las mismas imágenes) el intervalo del 95% del accuracy es [0.947, 0.963] para la red propia y [0.958, 0.972] para ResNet18, y el de la diferencia es [0.001, 0.019]. Como no incluye el 0 (la red propia empata o gana solo en 30 de los 2.000 sorteos), la ventaja de ResNet18 es real pero chica. Las dos redes además se equivocan en imágenes distintas: ResNet18 acierta 90 que la red propia erra, y la red propia acierta 62 que ResNet18 erra. Este intervalo mide la variación por las imágenes de test, no por volver a entrenar con otra semilla, que no medimos.

Tiempo por época. Medidas en la misma computadora y con la misma función de entrenamiento, una época de ResNet18 tarda 3.8 veces más que una de la red propia, lo que tiene sentido porque entrena más de 7 veces más parámetros. Pero como necesita muchas menos épocas, llega a 0.95 de accuracy de validation en la mitad del tiempo total (unos 7 minutos contra 13).

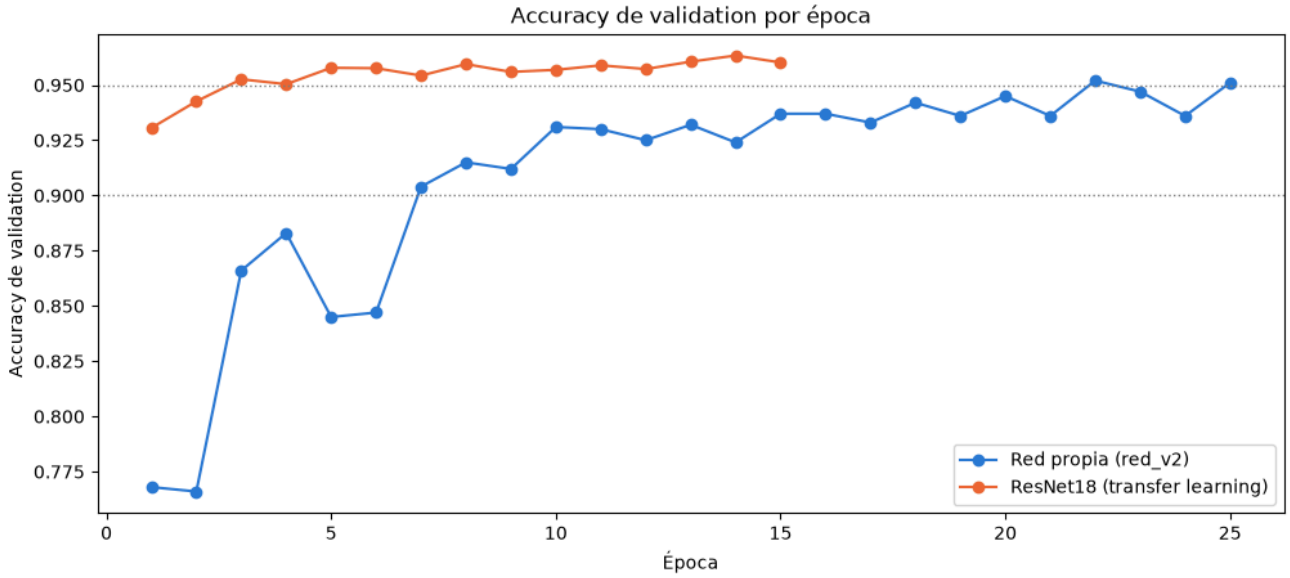


Figura 5. Accuracy de validation por época de las dos redes. Las líneas punteadas marcan 0.90 y 0.95.

5. Conclusiones y limitaciones

- Una CNN chica, de 3 bloques convolucionales y entrenada desde cero, ya llega a 95.5% de accuracy en test. Los errores que quedan se explican por clases que a esta resolución se ven parecidas.
- El espacio latente muestra que la red aprende a agrupar las imágenes por clase antes de la capa final (kNN 0.957 contra 0.346 con píxeles), y que los pares que más confunde son los que quedan pegados en esa representación.

- El transfer learning con ResNet18 converge en muchas menos épocas, termina un poco mejor (diferencia de accuracy con intervalo del 95% [0.001, 0.019]) y llega antes a un buen resultado, a cambio de una red diez veces más grande y de depender de un preentrenamiento en otro dataset.

Limitaciones.

- Probamos una sola configuración de la red propia (batch de 64, learning rate 0.001, 128 neuronas ocultas, el augmentation descrito) sin compararla con otras, por ejemplo sin augmentation. Cada entrenamiento tardaba entre 25 y 45 minutos en laptops sin GPU.
- Entrenamos cada red una sola vez: el bootstrap no mide cuánto cambiarían los resultados con otra semilla de entrenamiento.
- Dejamos las imágenes en 64 × 64 aunque ResNet18 se entrenó con 224 × 224, por el costo en CPU. A su última capa convolucional le llegan mapas de 2 × 2, así que agrandarlas o descongelar más capas probablemente mejoraría el resultado.
- Los tiempos son de una sola computadora: sirven para comparar las dos redes entre sí, pero no como valor absoluto.

Cómo correr el notebook

El notebook usa la carpeta `EuroSAT/`, que está al lado de `tp3.ipynb` y tiene el dataset de Kaggle (las imágenes, `train.csv`, `validation.csv`, `test.csv` y `label_map.json`). Junto al notebook están los pesos ya entrenados: `red_v2.pt` (red propia), `resnet18_tl.pt` y `historia_tl.json` (ResNet18). Con ellos, las celdas que los cargan recuperan los modelos sin reentrenar, aunque igual hay que correr antes las celdas que preparan los datos y definen las redes. Las celdas de entrenamiento siguen en el notebook con sus salidas originales.